The authoring flow

> **Superseded for the side bar layout.** The Docs side bar (Pages, Needs You, Sources, Publish) is defined in redesign.md. This page remains the record for the screenshot capture stage only.

The design for **screenshots in the authoring loop**, and for the side bar that had to change because of them. This is the record the next round argues with, so that a redesign starts from a decision rather than from the previous screenshot.

The reference

The bar is **Dr.Explain**, and its loop is not "a documentation editor with an image button":

> capture a *running window* → its controls become numbered callouts → write one line of text per number → export the same topics to CHM, PDF and the web.

Two properties make that work: the screenshot is taken *of a window*, so the callouts can be placed by the machine instead of by hand; and every artifact is built from one set of topics. Before this change we had no capture stage at all, so the side bar had filled the space with destinations that were not places in the documentation — Projects, Areas, Pipeline, Working Set.

The flow

```text
T3000 is running — the thing being documented
    │  Ctrl+Alt+S  capture a screen          Ctrl+Alt+V  paste from the clipboard
    ▼
CAPTURE     the screen becomes a file
    ▼
TOPIC       the page being written; the file lands in that page's own media/
    │       NN-area-slug.png · ≥ 1600 px · ≤ 2 MB · type sniffed, never trusted from th
    ▼
TEXT        the body, with the reference at the cursor
    ▼
GATE        registry · links · media audit · width · the version a page describes
    ▼
BUILD       the product repo's own scripts → the site, and the offline help
    ▼
NEW VERSION version-report.json → "retake these" rows → Ctrl+Alt+V again
```

Stages 1 and 3–5 exist and are untouched by this change. **Capture was the missing stage**, and it is now built as far as it can be without the callout decision below.

The rule that places every feature

The question a feature answers	Where it lives
"Where in the docs am I?"	the **Pages** tree
"What am I writing right now?"	the **page** and the editor's toolbar
"What is waiting for me?"	**Needs You**
"What am I shipping?"	one Build & Release page, reached from the tree's title bar
none of the above	it does not belong in the product

What changed

The tree gained a home of its own; the Documentation panel kept the groups it had.

Surface	What it is
Documentation panel (`temco`)	Projects · Pages · Areas · Pipeline · Working Set — as before
Documentation Tree (new activity-bar container, `temcoDocsTree`)	**Pages** — the documentation tree, the same provider as the panel's Pages view
	Needs You — the queue below it: screenshots with no page yet, pages the code moved away from, pages never reviewed

One provider behind both tree views, so there is still one answer to "what is the documentation". The menus on the panel's Pages view apply to the tree view as well.

Needs You lists only facts something else already computes, and every row runs a command that already existed:

- screenshots with no page yet, from the capture inbox;
- pages the product repo marks `stale` (read, never recomputed);
- pages never reviewed, as one row with a count and the report behind it.

Screenshots are a stage. `temcoDocs.captureImage` (Ctrl+Alt+S) opens the Windows snip overlay and takes what it puts on the clipboard; `temcoDocs.pasteImage` (Ctrl+Alt+V) takes an image that is already there. Both then:

1. find the page in front — the active editor's page, else the row the tree has selected;
2. write into that page's `media/` as `NN-slug.ext` , the number continuing the folder's own sequence;
3. insert `![title](media/NN-slug.ext)` at the cursor, or at the end of the page when it was not open;
4. warn — not refuse — below 1600 px wide, and refuse outright above 2 MB.

With no page in front the screen is not guessed at: it goes to `<root>/.temco/captures/` and waits in **Needs You** until *Attach the Screenshot to a Page…* puts it where a person says. The inbox is outside every `media/` , so the site only ever holds what a page shows.

`temcoDocs.chooseImage` is the third path, for a file that is already on disk; it is the one that also works outside Windows.

Why reading the clipboard needed no change to the core

The extension API reads the clipboard's **text** and not its image — `docs/temco-platform/ui/editor-design.md` §3 said so, and it is still true. But this application is Windows, and `System.Windows.Forms.Clipboard` reads the image with no native module and nothing installed: what Win+Shift+S, PrintScreen, Snipping Tool and every other capture tool leave behind. So the limit is a platform fact rather than a wall, and it costs one PowerShell child process per capture.

What is not built, and the decision it waits for

Annotation. Crop, arrow, box, text and blur/redact, saved over the capture, with the untouched original kept aside so a retake is diffable. It is a contained piece (a canvas over the captured image), but it contradicts `15-online-editor.md`, *What is not built* — "a screenshot is captured properly or not at all" — so it is a rule change that needs to be taken deliberately, not slipped in with a button.

Automatic callouts, the Dr.Explain core. It depends entirely on what is captured:

What the screen is of	What can be placed automatically
our own webview T3000	callouts from the real DOM elements — this is Dr.Explain's magic, nearly free
the classic Win32 T3000	control bounds from Windows UI Automation — automatic, but coarse
a device screen, or a customer's machine	nothing; manual crop, arrow, box, blur is the whole feature

The answer decides whether the annotator is a callout tool or a drawing tool, which is why it comes before the annotator.

The version report. `docs/help/version-report.json` — version, changed areas, affected pages, images older than the version they describe — written by the release workflow, read by **Needs You** and by Build & Release, consumed by the MCP server. Designed, not built; it needs no UI of its own.

Order

1. **The two views** — done.
2. **Capture, paste and the inbox** — done.
3. **The callout decision**, then annotation in the page.
4. **Build & Release** as one page: the build, review and release commands that exist today, in one place.
5. **`version-report.json`**, then the Action and the MCP wrapper.